SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures **COURSE CODE: CSA03**

Register Number	**1924260363**	
Name	**Eniya.E**	

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.* (BL2, BL3)

Assessment Tool Weightage: 35%

Questions:2 **EACH QUESTIONS CARRIES:** 10 MARKS **Total Marks:**20 **Duration:** ONE HOUR

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Code of Conduct:

I Eniya.E(1924260363) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

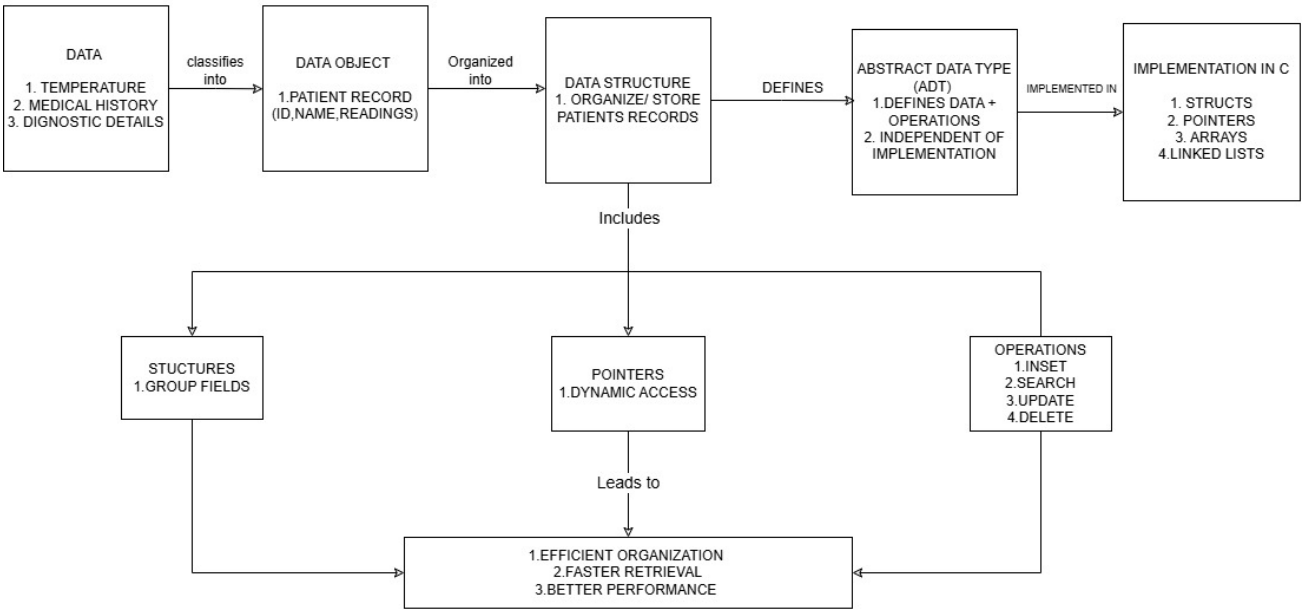
Signature of the student:
eniya.e

RUBRICS FOR CALCULATION:

Concept Mapping					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q1. SMART HEALTHCARE SYSTEM



DATA

Data refers to raw facts and values collected by a system. In a smart healthcare system, data may include patient temperature readings, medical history, diagnostic details, patient identification numbers, age, blood pressure readings, and other health-related information.

Data is the basic input required for storing, processing, and analyzing patient information. Proper organization of this data is essential for efficient healthcare management.

DATA OBJECT

A Data Object is a collection or representation of related data values that describe a particular entity. In a healthcare system, a patient can be considered an important data object.

For example, a patient's record may contain patient ID, age, temperature, medical history, and diagnostic information. These related values can be grouped together to represent one patient's complete data object.

Grouping related values into a data object makes the information easier to organize, manage, access, and process.

DATA STRUCTURE

A Data Structure is a systematic method of organizing and storing data so that it can be accessed and processed efficiently.

Common examples of data structures include arrays, linked lists, stacks, queues, trees, and graphs.

In a smart healthcare system, an array can be used to store temperature readings collected from several patients. A linked list can be used when patient records need to be dynamically added or removed.

The selection of an appropriate data structure depends on the type of data and the operations that need to be performed. A suitable data structure can improve the speed and efficiency of data processing.

ABSTRACT DATA TYPE (ADT)

An Abstract Data Type, or ADT, is a logical description of data and the operations that can be performed on that data without specifying how the operations are implemented.

An ADT focuses on what operations can be performed rather than how those operations are carried out. For example, a Queue ADT can provide operations such as Enqueue, Dequeue, Peek, and Is Empty. The Queue ADT can be implemented using different data structures, such as an array or a linked list.

RELATIONSHIP BETWEEN DATA STRUCTURE AND ADT

An ADT defines the logical behavior and operations of a data type, whereas a data structure provides the actual method of organizing the data in memory.

Therefore, an ADT describes what the system should do, while the data structure and implementation determine how it is done. For example, a Queue ADT specifies operations such as Enqueue and Dequeue. The queue can then be implemented using an array or a linked list.

IMPLEMENTATION IN C

C provides several features for implementing data structures and ADTs. Important features include structures, pointers, arrays, and functions.

These features allow programmers to create, store, access, and manipulate complex data efficiently.

STRUCTURES IN C

A structure in C is used to group related data members of different data types into a single unit. Structures are useful for representing real-world entities such as patients because a patient record can contain different types of information.

POINTERS

A pointer is a variable that stores the memory address of another variable.

Pointers are particularly important for implementing dynamic data structures such as linked lists, trees, and graphs.

Pointers allow different elements of a data structure to be connected. They can also support dynamic memory allocation when memory needs to be allocated during program execution.

For example, in a linked list, a pointer can connect one patient record to the next patient record.

OPERATIONS

Operations are the actions performed on data or data structures.

Common operations include:

Insertion – Adding a new data element.

Deletion – Removing an existing data element.

Searching – Finding a particular data element.

Sorting – Arranging data elements in a specific order.

Updating – Modifying existing information.

Accessing – Retrieving stored information.

In a healthcare system, these operations can be used to add a new patient record, search for a patient's information, update medical details, or remove an outdated record.

TIME COMPLEXITY

Time complexity describes the amount of time required to perform an operation as the amount of data increases.

Selecting a suitable data structure can reduce the time required for operations such as searching, insertion, deletion, and accessing data.

Therefore, time complexity is an important factor when selecting a suitable data structure for a healthcare system.

SPACE COMPLEXITY

Space complexity describes the amount of memory required by a data structure or an algorithm.

An efficient data structure should not only process information effectively but should also use memory efficiently.

Therefore, both time complexity and space complexity should be considered when designing a healthcare data-management system.

IMPORTANCE IN SMART HEALTHCARE

A smart healthcare system processes large volumes of patient information. Efficient data structures help the system store, access, update, and process this information effectively.

For example, patient records can be represented using structures, pointers can support dynamic data structures, and appropriate operations can be implemented using functions.

The correct selection of a data structure can improve processing speed, memory utilization, and overall system performance.

PROGRESSION OF CONCEPTS

The progression can be represented as:

Data → Data Object → Data Structure → Abstract Data Type → Implementation in C

Raw healthcare data is first represented as meaningful data objects. These objects are organized using suitable data structures. The ADT defines the logical operations and behavior of the data. Finally, C provides programming features such as structures, pointers, arrays, and functions for implementation.

CONTRIBUTION TO SYSTEM PERFORMANCE

Each level contributes to efficient system performance.

Data provides the basic information required by the system. Data objects organize related information into meaningful units. Data structures determine how the information is stored and accessed. ADTs define

the operations that can be performed on the data. C implementation provides the actual programming mechanisms required to execute these operations. The appropriate selection of data structures can reduce processing time and improve memory utilization. This is especially important in healthcare systems because they need to manage large volumes of patient information.

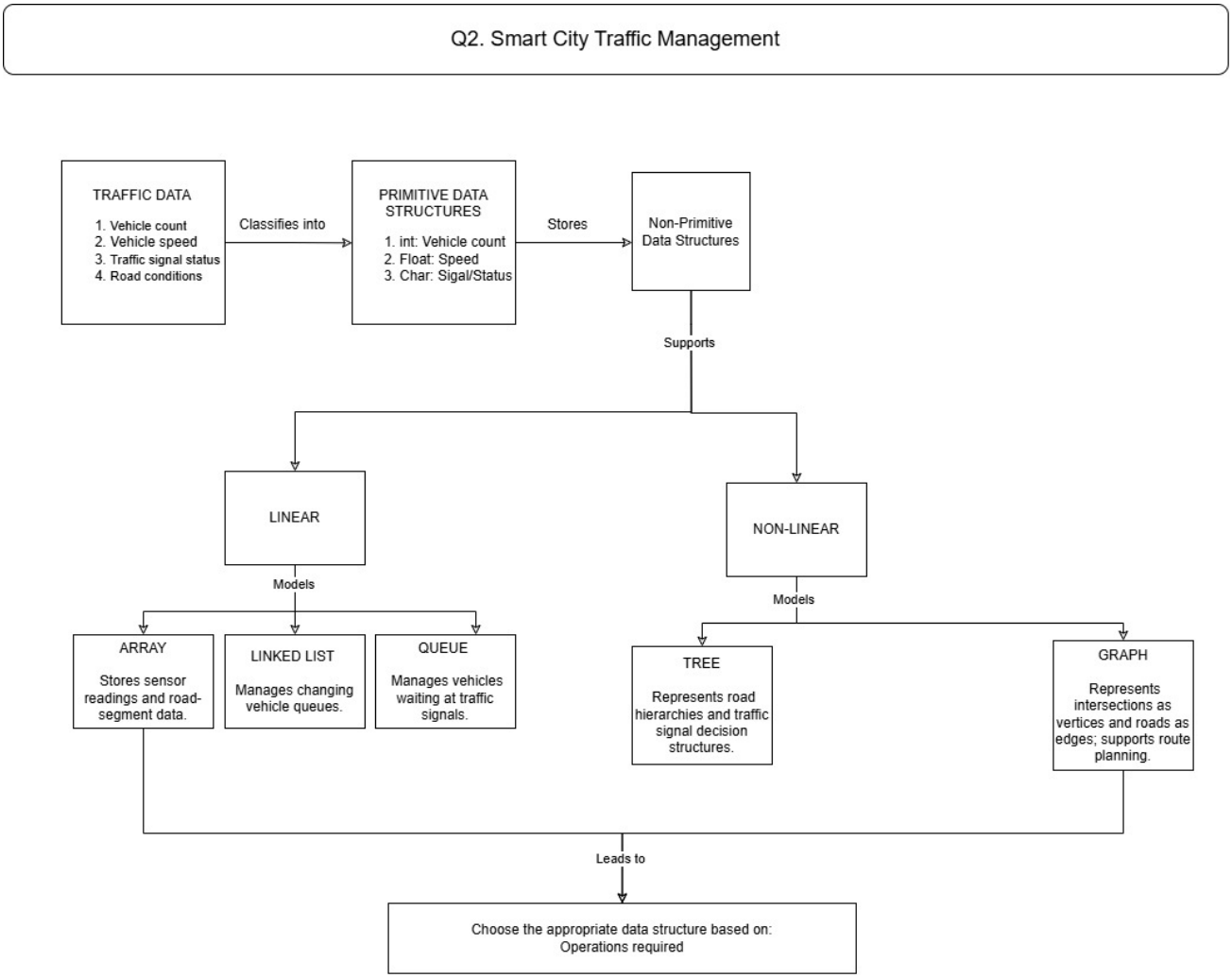
CONCLUSION

The progression from Data to Data Object, Data Structure, Abstract Data Type, and finally Implementation in C provides a systematic approach to organizing and processing healthcare information.

The use of suitable structures, pointers, and operations allows large amounts of patient data to be stored and processed efficiently. Considering time complexity and space complexity while selecting data structures further improves system performance and memory utilization.

Therefore, appropriate data organization and implementation are essential for developing an efficient smart healthcare system.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.



DATA STRUCTURE

Data structures are methods of organizing and storing data so that it can be accessed and processed efficiently. In a smart city traffic management system, data structures are used to manage information related to vehicles, roads, junctions, traffic signals, and routes.

PRIMITIVE DATA STRUCTURES

Primitive data structures are the basic data types provided by a programming language. They represent individual values. Examples include integer, float, character, and Boolean. In traffic management, an integer can represent the number of vehicles or signal duration, a float can represent vehicle speed or distance, a character can represent a vehicle category or signal status, and a Boolean can represent whether a traffic signal is ON or OFF.

NON-PRIMITIVE DATA STRUCTURES

Non-primitive data structures are complex structures used to store collections of data and represent relationships between data elements. They are mainly classified into linear and non-linear data structures.

LINEAR DATA STRUCTURES

Linear data structures organize elements in a sequential order. Common examples include arrays, linked lists, stacks, and queues.

ARRAY

An array stores elements of the same type in a sequence and provides direct access using an index. In traffic management, arrays can be used to store vehicle counts at different junctions, traffic signal status, or traffic readings collected at different time intervals.

LINKED LIST

A linked list consists of nodes, where each node contains data and a link to another node. In traffic management, linked lists can be used to manage dynamically changing vehicle or road-related records where elements are frequently added or removed.

STACK

A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle. It can be used for certain route-processing or system operations where the most recently stored information needs to be processed first.

QUEUE

A queue is a linear data structure that follows the First-In, First-Out (FIFO) principle. It can represent vehicles waiting at a traffic signal, where vehicles are processed according to their order of arrival.

NON-LINEAR DATA STRUCTURES

Non-linear data structures do not arrange elements in a single sequential order. They represent hierarchical or interconnected relationships. The main examples are trees and graphs.

TREE

A tree is a non-linear data structure used to represent hierarchical relationships between elements. In traffic management, a tree can represent information such as City → Zones → Areas → Roads. Therefore, trees are useful for organizing hierarchical traffic and road information.

GRAPH

A graph consists of vertices and edges. Vertices can represent locations or junctions, while edges can represent roads or connections between them. Graphs are highly suitable for representing city road networks. For example, vertices can represent junctions, edges can represent roads, and edge values can represent distance, travel time, or traffic cost.

CLASSIFICATION BASED ON FUNCTIONALITY AND EFFICIENCY

Primitive structures are used for basic individual values, while non-primitive structures are used for complex collections and relationships. Linear structures organize data sequentially, making them suitable for arrays, linked lists, stacks, and queues. Non-linear structures represent hierarchical or interconnected relationships, making them suitable for trees and graphs. The efficiency of a data structure depends on the operation required. Arrays provide fast